Árvores balanceadas na Black Friday

AVL ou Rubro-Negra: qual usar para registrar, achar e remover encomendas?

LETÍCIA QUEIROZ WANDERLEY

IMD0039 · ESTRUTURAS DE DADOS BÁSICAS II

Sumário

A proposta 3

O desafio técnico 4

A solução 5

Árvore AVL 6

Árvore Rubro-Negra 7

Demonstração 8

Testes realizados 9

Análise comparativa 10

Recomendação 11

Referências 12

A proposta

Na Black Friday o sistema de logística recebe, consulta e apaga identificadores de encomendas o tempo todo. Se a estrutura de dados engasgar, o pedido atrasa.

Objetivo: implementar AVL e Rubro-Negra do zero, rodar os mesmos testes nas duas e defender uma recomendação com base nos números.



Registrar

INSERIR O ID NA CHEGADA



Localizar

BUSCAR SEM VARRER TUDO



Remover

APAGAR NO FIM DA ENTREGA

O desafio técnico

A ORDEM DE CHEGADA DEFINE O FORMATO

Numa árvore de busca comum, ids em sequência, o que é normal em pedidos, viram uma lista encadeada disfarçada. A altura cresce até n e cada busca fica cada vez mais lenta.

100.000

COMPARAÇÕES NO PIOR CASO, SEM BALANCEAMENTO

17 a 30

NÍVEIS COM BALANCEAMENTO, NOS NOSSOS TESTES

A solução desenvolvida

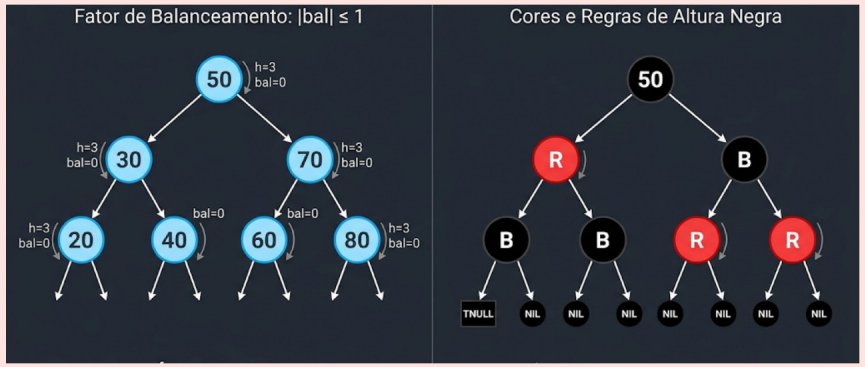
Implementei para a equipe Amazon

AVL e Rubro-Negra escritas à mão, com inserção, remoção, cálculo de altura e contador de rotações.

Decisões tomadas

Valor repetido é ignorado na inserção.
Remover um valor que não existe não muda a árvore.

Árvores AVL e Rubro-Negra

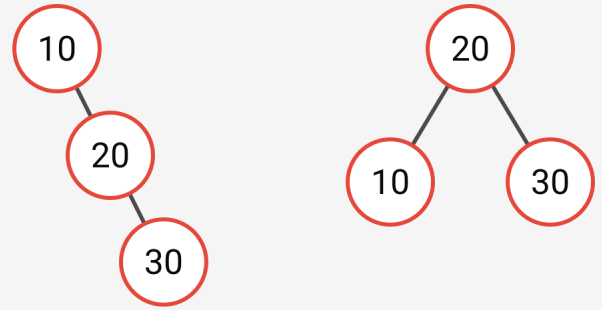


Árvore AVL

Cada nó guarda sua altura. O fator de balanceamento é a altura da esquerda menos a da direita e tem que ficar entre −1 e 1.

Se passar disso, a árvore gira: simples à direita (LL), simples à esquerda (RR) ou dupla (LR e RL).

Exemplo: inserir 10, 20 e 30. O 10 fica com fator −2, e uma rotação à esquerda põe o 20 na raiz.



antes (fator −2) depois

Árvore Rubro-Negra

- A raiz é preta.
- Nó vermelho só tem filhos pretos.
- Todo caminho até uma folha passa pelo mesmo número de nós pretos.
- As folhas (TNULL) são pretas.
- Todo nó novo entra vermelho.

Exemplo: inserir 10, 20 e 30

O 20 entra vermelho sob o 10 e o 30 entra vermelho sob o 20: dois vermelhos seguidos e tio preto. Rotação à esquerda no 10, o 20 vira raiz preta e os outros dois ficam vermelhos. Se o tio fosse vermelho, bastava recolorir.

Aqui a RB permite altura até $2 \cdot \log_2(n+1)$, bem mais frouxa que a AVL.

Demonstração do sistema

EXECUÇÃO COM 10.000 ELEMENTOS

Na inserção crescente a AVL fez 9.470 rotações e terminou com 14 níveis. A Rubro-Negra fez 9.460 e terminou com 24.

Duplicatas e remoções de valores inexistentes não geram nenhuma rotação.

Saída real do programa. A AVL imprime altura em arestas e a RB em nós.

```
==============================================
INICIANDO BATERIA DE TESTES - 10000 ELEMENTOS
==============================================

> CENÁRIO 1: INSERÇÃO CRESCENTE
AVL   -> Tempo: 20   ms | Rotações: 9470     | Altura Final: 13
RB    -> Tempo: 9    ms | Rotações: 9460     | Altura Final: 24

> CENÁRIO 2: INSERÇÃO DECRESCENTE
AVL   -> Tempo: 19   ms | Rotações: 9470     | Altura Final: 13
RB    -> Tempo: 8    ms | Rotações: 9460     | Altura Final: 24

> CENÁRIO 3: INSERÇÃO ALEATÓRIA
AVL   -> Tempo: 26   ms | Rotações: 6558     | Altura Final: 15
RB    -> Tempo: 9    ms | Rotações: 5628     | Altura Final: 16

> CENÁRIO 4: INSERÇÃO DE DUPLICATAS (10% dos elementos)
AVL   -> Tempo: 1    ms | Rotações: 0        | Altura Final: 15
RB    -> Tempo: 0    ms | Rotações: 0        | Altura Final: 16

> CENÁRIO 5: REMOÇÃO DE ELEMENTOS INEXISTENTES (-1 a -100)
AVL   -> Tempo: 0    ms | Rotações: 0        | Altura Final: 15
RB    -> Tempo: 0    ms | Rotações: 0        | Altura Final: 16
```

Testes realizados

1

Inserção crescente

Pior caso para uma BST comum

2

Inserção decrescente

Mesmo problema, no sentido oposto

3

Inserção aleatória

Comportamento médio

4

Duplicatas (10%)

Valores repetidos são ignorados

5

Remoção de inexistentes

Nada muda, nenhuma rotação

6

Remoção aleatória (50%)

Rebalanceamento após remover

Mesmos vetores nas duas árvores (semente 42), com 1.000, 10.000 e 100.000 elementos. Ainda falta: árvore vazia, um só elemento e checagem das propriedades.

Análise comparativa (100.000 elementos)

Cenário	Rotações AVL	Rotações RB	Níveis AVL	Níveis RB	Tempo AVL	Tempo RB
Inserção crescente	94.988	94.975	17	30	16 ms	14 ms
Inserção aleatória	66.275	55.304	20	20	33 ms	23 ms
Remoção de 50%	21.069	21.992	18	20	16 ms	13 ms

Com dados ordenados a RB faz quase as mesmas rotações, mas termina com 30 níveis contra 17. Na inserção aleatória ela gira cerca de 17% menos. Na remoção, a AVL ficou mais baixa.

Níveis da AVL = altura em arestas + 1, para igualar à contagem da RB. Tempos de uma única execução, em milissegundos.

RECOMENDAÇÃO TÉCNICA

Minha escolha para o cenário: AVL

- Encomendas costumam chegar em sequência, e nesse caso a AVL ficou com 17 níveis contra 30 da RB. Buscas mais curtas.
- O custo extra é pequeno: 16 ms contra 14 ms na inserção ordenada.
- Se a carga virar quase só escrita (inserir e remover), a RB gira menos e insere mais rápido, e passa a valer a pena.

Limite: uma execução por cenário e tempo em ms. Para cravar o ganho em tempo, repetir as medições.

Referências e repositório

Código: https://github.com/elektrawanq/amazon-engineering-challenge-avl-rb.git
Autor: Letícia Queiroz Wanderley

- CORMEN, T. et al. Algoritmos: Teoria e Prática. Elsevier.
- SEDGEWICK, R.; WAYNE, K. Algorithms. Addison-Wesley.
- ADELSON-VELSKY, G.; LANDIS, E. An algorithm for the organization of information, 1962.
- GUIBAS, L.; SEDGEWICK, R. A dichromatic framework for balanced trees, 1978.

Obrigada!

Amazon Engineering Challenge

LETÍCIA QUEIROZ WANDERLEY